Buildable

{final} uuid: S tring
crimeRate: int
visitors: List<Citizen>
isDestroyed: boolean
buildables: List<Class<? extends Buildable>>
registry: Map<Class<? extends Buildable>, Supplier<Buildable>>
blobRegistry: Map<Class<? extends Buildable>, BiFunction<SerializedBlob, City, Buildable>>

Buildable()
Buildable(blob: SerializedBlob, city: City)
toBlob(factory: SerializedBlob.Factory): SerializedBlob
getVisitors(): List<Citizen>
addVisitor(citizen: Citizen): void
removeVisitor(citizen: Citizen): void
runSimulationTick(city: City): void
getW idth(): int
getLength(): int
getPrice(): int
destroy(): void
getIsDestroyed(): boolean
getCrimeRate(): int
getDetailedInfo(): M ap<String, String>
createBuilding(Class<? extends Buildable>): Buildable
fromBlob(blob: SerializedBlob, city: City): Buildable

Buildable.UnregisteredBuildingType

UnregisteredBuildingType(Class<? extends Buildable>)

Exception

Buildable.BuildableRef

T

{fin a l} city: C ity
{final} buildableId: S tring
{final} buildableType: String
cache: Map<BuildableRef<?>, Buildable>

BuildableRef(buildable: Buildable, city: City)
BuildableRef(blob: SerializedBlob, city: City)
toBlob(factory: SerializedBlob.Factory): SerializedBlob
get(): T
upcast(): BuildableRef<Buildable>

PrivateBuilding

isBuilt: boolean

getIsBuilt(): boolean
build(): void
calculateProfitPerTick(): int
calculateDemand(building: PrivateBuilding, city: City): int

ResidentialBuilding

{final} RESIDENT_TAX: int
residents: ArrayList<Citizen>

getCapacity(): int
getResidents(): List<Citizen>
addResident(citizen: Citizen): void
evictResidents(): List<Citizen>
destroy(): void
getResidentTax(): int
calculateProfitPerTick(): int
calculateDemand(city: City): int
calculateHousingShortage(city: City): double

SmallHouse

getPrice(): int
getCapacity(): int
getW idth(): int
getLength(): int

Condominium

getPrice(): int
getCapacity(): int
getW idth(): int
getLength(): int

WorkplaceBuilding

{final} BUSINESS_TAX: int
hiredW orkers: ArrayList<Citizen>

getHiredW orkers(): List<Citizen>
addHiredWorker(worker: Citizen): void
removeHiredWorker(worker: Citizen): void
destroy(): void
getBusinessTax(): int
calculateProfitPerTick(): int
getWorkersCapacity(): int
hasOpenJobPositions(): boolean
calculateLaborShortage(city: City): double

IndustrialBuilding

calculateDemand(city: City): int

Factory

getPrice(): int
getW orkersCapacity(): int
getW idth(): int
getLength(): int

Warehouse

getPrice(): int
getW orkersCapacity(): int
getW idth(): int
getLength(): int

OfficeBuilding

calculateDemand(city: City): int

Skyrise

getPrice(): int
getW orkersCapacity(): int
getW idth(): int
getLength(): int

BankBranch

getPrice(): int
getW orkersCapacity(): int
getW idth(): int
getLength(): int

CommercialBuilding

{final} SALES_TAX: int

getVisitorsCapacity(): int
addVisitor(citizen: Citizen): void
calculateSalesTax(): int
calculateProfitPerTick(): int
calculateDemand(city: City): int
calculateRetailShortage(city: City): double

Mall

getPrice(): int
getW orkersCapacity(): int
getVisitorsCapacity(): int
getW idth(): int
getLength(): int

GroceryStore

getPrice(): int
getW orkersCapacity(): int
getVisitorsCapacity(): int
getW idth(): int
getLength(): int

PublicBuilding

getMaintanenceCostPerDay(): int
getUpgrades(): Upgrade[]

PublicBuilding.Upgrade

isBuilt: boolean

getPrice(): int
getName(): String
getIsBuilt(): boolean
build(city: City): void
toBlob(factory: SerializedBlob.Factory): SerializedBlob

PublicServiceBuilding

getRange(): int
getPublicServiceTypeClass(): Class<? extends PublicServiceBuilding>
getFieldFunctionForPublicServiceType(city: City, type: Class): Function<Point, Double>
getAllPublicServiceTypeBuildings(city: City, type: Class): Map<Point, PublicServiceBuilding>
getCombinedFieldFunctionForBuildings(buildings: Map): Function<Point, Double>
getFieldFunctionForBuilding(origin: Point, building: PublicServiceBuilding): Function<Point, Double>

HealthcareBuilding

getHealthIncrease(x: int, y: int): int
getPublicServiceTypeClass(): Class<? extends PublicServiceBuilding>

Clinic

ambulanceGarage: AmbulanceGarage

getPrice(): int
getUpgrades(): Upgrade[]
getMaintanenceCostPerDay(): int
getRange(): int
getHealthIncrease(x: int, y: int): int
getW idth(): int
getLength(): int

Clinic.AmbulanceGarage

getPrice(): int

Hospital

helipad: Helipad

getPrice(): int
getUpgrades(): Upgrade[]
getMaintanenceCostPerDay(): int
getRange(): int
getHealthIncrease(x: int, y: int): int
getW idth(): int
getLength(): int

Hospital.Helipad

getPrice(): int

PoliceStation

getCrimeReduction(x: int, y: int): int
getPublicServiceTypeClass(): Class<? extends PublicServiceBuilding>

SmallPoliceStation

policeCarsGarage: PoliceCarsGarage

getPrice(): int
getUpgrades(): Upgrade[]
getMaintanenceCostPerDay(): int
getRange(): int
getCrimeReduction(x: int, y: int): int
getW idth(): int
getLength(): int

SmallPoliceStation.PoliceCarsGarage

getPrice(): int

BigPoliceStation

helipad: Helipad

getPrice(): int
getUpgrades(): Upgrade[]
getMaintanenceCostPerDay(): int
getRange(): int
getCrimeReduction(x: int, y: int): int
getW idth(): int
getLength(): int

BigPoliceStation.Helipad

getPrice(): int

PublicTransportation

getCapacity(): int
computeNoiseLevel(): int
getCongestion(): int

Road

plantedTrees: PlantedTrees

getUpgrades(): Upgrade[]
getPrice(): int
getMaintanenceCostPerDay(): int
getCapacity(): int
computeNoiseLevel(): int
getW idth(): int
getLength(): int

Road.PlantedTrees

getPrice(): int

Street

speedLimiters: SpeedLimiters
plantedTrees: PlantedTrees

getUpgrades(): Upgrade[]
getPrice(): int
getMaintanenceCostPerDay(): int
getCapacity(): int
computeNoiseLevel(): int
getW idth(): int
getLength(): int

Street.SpeedLimiters

getPrice(): int

Street.PlantedTrees

getPrice(): int

C ity

{final} grid: C ityG rid
loans: ReactiveArrayList<Loan>
{final} loansView: O bservable<List<Loan>>
m oney: Reactive<Double>
{final} m oneyView: O bservable<Double>
{final} nam e: S tring
homelessPeople: ArrayList<Citizen>
DEMOLISHMENT_COEF: double
TICKS_IN_DAY: int

City(sizeX: int, sizeY: int, startingMoney: double, name: String)
City(blob: SerializedBlob)
toBlob(factory: SerializedBlob.Factory): SerializedBlob
build(upgrade: PublicBuilding.Upgrade): boolean
build(building: Buildable, place: Point): void
demolish(place: Point): void
takeOutLoan(loan: Loan): void
payOutLoan(loan: Loan): void
serviceLoans(): double
getTotalDebt(): double
payBuildingsUpkeepPerTick(): int
accomodateHomelessPeople(): void
builtBuildings(): List<PrivateBuilding>
collectTaxesFromResidents(): int
collectTaxesFromPurchases(): int
collectTaxesFromBusinesses(): int
evictHomelessPeople(): void
employPeople(): void
accomodateNewImmigrants(): void

CityGrid

{final} buildings: M ap<Point, Buildable>
{final} buildingsView: M ap<Point, Buildable>
{final} sizeX: int
{final} sizeY: int
pathFinder: PathFinder

CityGrid(sizeX: int, sizeY: int)
CityGrid(blob: SerializedBlob, city: City)
toBlob(factory: SerializedBlob.Factory): SerializedBlob
getNextStepFromTo(from: Buildable, to: Buildable): Buildable
getBuildingAt(place: Point): Buildable
getBuildingOrigin(building: Buildable): Point
placeBuildingAt(place: Point, building: Buildable): void
removeBuildingAt(place: Point): void

Citizen

{fin a l} id: S trin g
location: BuildableRef<Buildable>
home: BuildableRef<ResidentialBuilding>
work: BuildableRef<W orkplaceBuilding>
currentHealth: int
{fin a l} id e n tity: Id e n tity
lastStateUpdateTick: int
city: C ity
currentThoughtsSeed: int
state: CitizenState
cache: WeakHashMap<String, Citizen>

Citizen(city: City, home: ResidentialBuilding, work: WorkplaceBuilding)
evict(): void
toBlob(factory: SerializedBlob.Factory): SerializedBlob
fromBlob(blob: SerializedBlob, city: City): Citizen
runSim ulationTick(tick: int): void
getLocation(): Buildable
getCurrentHealth(): int
setCurrentHealth(health: int): void
getSatisfaction(): int
getDetailedInfo(): M ap<String, String>
getCurrentStateDescription(): String
getCurrentThoughts(): String

«sealed»
CitizenState

toBlob(factory: SerializedBlob.Factory): SerializedBlob
fromBlob(blob: SerializedBlob, city: City): CitizenState

«record»

CitizenState.Residing

«record»

CitizenState.Working

«record»

CitizenState.Shopping

shop: BuildableRef<CommercialBuilding>

toBlob(factory: SerializedBlob.Factory): SerializedBlob

«final»

IdentityGenerator

FIRST_NAMES_MALE: List<String>
FIRST_NAMES_FEMALE: List<String>
LAST_NAMES: List<String>
DEFAULT_MIN_YEAR: int
DEFAULT_MAX_YEAR: int
{final} random: R andom
{final} m inY ear: int
{final} m axY ear: int
generator: IdentityGenerator

IdentityGenerator()
IdentityGenerator(seed: long)
IdentityGenerator(random: Random, minYear: int, maxYear: int)
nextIdentity(): Identity

«record»

IdentityGenerator.Identity

firstNam e: String
lastName: String
birthYear: int

fullName(): String
toBlob(factory: SerializedBlob.Factory): SerializedBlob

PathFinder

{final} grid: C ityG rid
tileIndexCache: Map<Point, Buildable>
originCache: Map<Buildable, Point>
adjacencyCache: Map<Buildable, List<Buildable>>
flowFieldCache: Map<Buildable, Map<Buildable, Buildable>>

PathFinder(grid: CityGrid)
invalidateCache(): void
buildingAt(p: Point): Buildable
nextStep(from: Buildable, to: Buildable): Buildable
findPath(from: Buildable, to: Buildable): Deque<Buildable>
isStillWalkable(b: Buildable, target: Buildable): boolean

Loan

paym entLeft: Reactive<Integer>
paymentLeftView: Observable<Integer>

getLoanSize(): int
getPaymentLeft(): int
getLoanRate(): int
getPennyRate(): int
payOutLoan(money: int): void
paidOutLoan(): boolean
getPerTickInterest(): int
payPerTickInterest(money: int): void
compareTo(other: Loan): int
toBlob(factory: SerializedBlob.Factory): SerializedBlob
fromBlob(blob: SerializedBlob): Loan

FederalLoan

getLoanSize(): int
getLoanRate(): int
getPennyRate(): int

PrivateLoan

getLoanSize(): int
getLoanRate(): int
getPennyRate(): int

«java.lang»
Comparable

Loan

Game

{final} sim ulator: R eactive<Sim ulator>

listSaves(): List<String>
loadSave(gameName: String): void
createSave(): void
startNewSimulation(mapSizeX: int, mapSizeY: int, difficulty: GameDifficulty, cityName: String): void

GameDifficulty

EASY
MEDIUM
HARD

{final} startingM oney: int

GameSpeed

Stopped
Slow
Normal
Fast

{final} m sBetweenTicks: int

Simulator

{fin a l} city: C ity
gameSpeed: GameSpeed
currentTick: Reactive<Integer>
{final} currentTickView: O bservable<Integer>
citizensAm ount: Reactive<Integer>
citizensAmountView: Observable<Integer>
homelessCitizensAmount: Reactive<Integer>
homelessCitizensAmountView: Observable<Integer>
lastResidentTax: Reactive<Integer>
lastResidentTaxView: Observable<Integer>
lastBusinessTax: Reactive<Integer>
lastBusinessTaxView: Observable<Integer>
lastPurchaseTax: Reactive<Integer>
lastPurchaseTaxView: Observable<Integer>
lastBuildingsUpkeep: Reactive<Integer>
lastBuildingsUpkeepView: Observable<Integer>
netIncomeView: Observable<Integer>
lastLoansService: Reactive<Double>
lastLoansServiceView: Observable<Double>
totalDebt: Reactive<Double>
totalDebtView: Observable<Double>

Sim ulator(city: City)
getGameSpeed(): GameSpeed
setGameSpeed(gameSpeed: GameSpeed): void
startSim ulation(): void
startSimulationCLI(): Runnable
runSimulationTick(): void
averageCitizenSatisfaction(): int
averageCrimeRate(): int
averageCitizenHealth(): int

Point

{fin a l} x: in t
{fin a l} y: in t

Point(x: int, y: int)
Point(blob: SerializedBlob)
toBlob(factory: SerializedBlob.Factory): SerializedBlob
distFrom(other: Point): double
hashCode(): int
equals(obj: Object): boolean
isW ithin(from: Point, l: int, w: int): boolean
allPointsW ithin(from: Point, l: int, w: int): Iterable<Point>

Reactive

T

value: T
{final} listeners: CopyO nW riteArrayList<Consum er<T>>

Reactive()
Reactive(initial: T)
get(): T
set(newValue: T): void
subscribe(listener: Consumer<T>): Runnable
readOnly(): Observable<T>

Reactive.Observable

T

get(): T
subscribe(listener: Consumer<T>): Runnable

Reactive.ReactiveArrayList

T

{final} listeners: CopyO nW riteArrayList<Consum er<List<T>>>

add(e: T): boolean
remove(o: Object): boolean
addAll(c: Collection<? extends T>): boolean
clear(): void
set(index: int, element: T): T
get(): List<T>
subscribe(listener: Consumer<List<T>>): Runnable
readO nly(): O bservable<List<T>>

ArrayList

SerializedBlob

extendMap(m: Map<String, SerializedBlob>): SerializedBlob
type(): Factory.Type
map(): Map<String, SerializedBlob>
array(): List<SerializedBlob>
prim itive(): Object
string(): String
intValue(): Integer
booleanValue(): Boolean
doubleValue(): Double
toSerialized(): String

SerializedBlob.Factory

fromMap(m: Map<String, SerializedBlob>): SerializedBlob
array(l: List<SerializedBlob>): SerializedBlob
string(s: String): SerializedBlob
intValue(i: int): SerializedBlob
doubleValue(d: double): SerializedBlob
booleanValue(b: boolean): SerializedBlob
nullValue(): SerializedBlob
primitive(o: Object): SerializedBlob
parse(serialized: String): SerializedBlob

SerializedBlob.Factory.Type

OBJECT
ARRAY
PRIMITIVE

SelfDescribedXML

singleton: SelfDescribedXML

getInstance(): SelfDescribedXML
fromMap(m: Map<String, SerializedBlob>): SelfDescribedXMLBlob
array(l: List<SerializedBlob>): SelfDescribedXMLBlob
string(s: String): SelfDescribedXMLBlob
intValue(i: int): SelfDescribedXMLBlob
doubleValue(d: double): SelfDescribedXMLBlob
booleanValue(d: boolean): SelfDescribedXMLBlob
nullValue(): SelfDescribedXMLBlob
primitive(o: Object): SelfDescribedXMLBlob
parse(xml: String): SelfDescribedXMLBlob
toEl(blob: SerializedBlob, doc: Document): Element
from(el: Element): SelfDescribedXMLBlob

SelfDescribedXML.SelfDescribedXMLBlob

{final} type: T ype
map: Map<String, SerializedBlob>
{final} array: List<SerializedBlob>
{final} prim itive: O bject

extendMap(m: Map<String, SerializedBlob>): SerializedBlob
type(): Type
map(): Map<String, SerializedBlob>
array(): List<SerializedBlob>
prim itive(): Object
string(): String
intValue(): Integer
booleanValue(): Boolean
doubleValue(): Double
toSerialized(): String

Project

m ain(args: String[]): void

GameInterface

FRAME_W: int
FRAME_H: int

m ain(args: String[]): void
run(): void
setGlobalFont(): void
createMenuBar(): JMenuBar
render(): void

CLIInterface

RESET: String
BOLD: String
CYAN: String
GREEN: String
YELLOW: String
RED: String
GREY: String
WHITE: String

m ain(args: String[]): void
run(): void
c(colour: String, text: String): String
iconFor(b: Buildable): String
printMap(sim: Simulator): void
printStats(sim: Simulator): void
buildableList(): List<Class<? extends Buildable>>
printBuildingList(): void
printHelp(inGame: boolean): void
handleSimCommand(line: String, sim: Simulator, game: Game, sc: Scanner): boolean
runSimulationSession(game: Game): void
mainMenuLoop(game: Game, sc: Scanner): void

M ain

m ain(args: String[]): void

FarmCLI

RESET: String
BOLD: String
GREEN: String
YELLOW: String
CYAN: String
RED: String
BLUE: String
DIM: String
MAGENTA: String
{fin a l} fa rm: F a rm
{final} m arket: Farm M arket
m oney: int
day: int
{final} scanner: Scanner

FarmCLI(plots: int, startMoney: int)
run(): void
cm dPlant(parts: String[]): void
cmdTick(): void
cm dHarvest(parts: String[]): void
cm dSell(parts: String[]): void
cm dEvent(parts: String[]): void
cm dW ater(parts: String[]): void
printBanner(): void
printStatus(): void
printM arket(): void
printHelp(): void
validatePlotIndex(idx: int): void
findCropByName(name: String): Crop
m ain(args: String[]): void

FarmGUI

STAGE_COLORS: Color[]
CROP_OPTIONS: String[][]
{fin a l} fa rm: F a rm
{final} m arket: Farm M arket
m oney: int
day: int
selectedPlot: int
{final} onReturnToMain: Runnable
tilledDirtIm g: Im age
grassImg: Image
{final} plotPanels: PlotPanel[]
dayLabel: JLabel
moneyLabel: JLabel
waterLabel: JLabel
logArea: JTextArea
cropTypeBox: JComboBox<String>
selectedLabel: JLabel
plantBtn: JButton
tickBtn: JButton
harvestBtn: JButton
sellBtn: JButton
sellQtyField: JTextField
marketPanel: JPanel

FarmGUI(numPlots: int, startMoney: int, onReturnToMain: Runnable)
loadSprites(): void
loadRes(name: String): Image
getSpriteFor(crop: Crop, stage: GrowthStage): Image
getCropKey(crop: Crop): String
buildUI(): void
buildHeader(): JPanel
buildCenter(): JPanel
buildSidebar(): JPanel
buildActionPanel(): JPanel
buildCropLegend(): JPanel
buildMarketPanel(): JPanel
buildLog(): JPanel
doPlant(): void
doTick(): void
doHarvest(): void
doHarvestAll(): void
doSell(): void
doEvent(e: FarmEvent): void
refreshAll(): void
refreshMarket(): void
log(msg: String): void
miniEventBtn(label: String, event: FarmEvent): JButton
hudLabel(text: String): JLabel
styledButton(label: String, bg: Color): JButton
style(box: JComboBox<?>): void
style(tf: JTextField): void
rowLabel(text: String): JLabel
panelBorder(title: String): Border
selectPlot(i: int): void
m ain(args: String[]): void

farm.core.Crop

DEFAULT_MARKETPRICES_FOR_TYPES: Map<String, Integer>
stage: GrowthStage
marketPrice: int
nam e: String
cropType: String
fEvent: FarmEvent

Crop(name: String, marketPrice: int, cropType: String)
Crop(name: String, cropType: String)
getName(): String
getStage(): GrowthStage
setStage(gs: GrowthStage): void
FarmEventHappened(e: FarmEvent): void
getFEvent(): FarmEvent
grow(): void
isReady(): boolean
getYield(): int
getType(): String
getM arketprice(): int
appropriateEventYield(usualYield: int): int

farm.core.GrainCrop

DEFAULT_YIELD: int

GrainCrop(name: String, marketPrice: int)
GrainCrop(name: String)
getYield(): int
getType(): String

farm.core.VegetableCrop

DEFAULT_YIELD: int

VegetableCrop(name: String, marketPrice: int)
VegetableCrop(name: String)
getYield(): int
getType(): String

farm.core.OrchardCrop

DEFAULT_YIELD: int

OrchardCrop(name: String, marketPrice: int)
OrchardCrop(name: String)
getYield(): int
getType(): String

AppleCrop

TICKS_PER_STAGE: int
APPLE_YIELD: int
tickCounter: int

AppleCrop()
grow(): void
getYield(): int
getTicksPerStage(): int
getCropKey(): String

SunflowerCrop

TICKS_PER_STAGE: int
SUNFLOWER_YIELD: int
tickCounter: int

SunflowerCrop()
grow(): void
getYield(): int
getTicksPerStage(): int
getCropKey(): String

CornCrop

TICKS_PER_STAGE: int
CORN_YIELD: int
tickCounter: int

CornCrop()
grow(): void
getYield(): int
getTicksPerStage(): int
getCropKey(): String

WheatCrop

ticksToGrow: int
tickCounter: int
TICKS_PER_STAGE: int

WheatCrop()
grow(): void
getTicksPerStage(): int
getCropKey(): String

PotatoCrop

TICKS_PER_STAGE: int
POTATO_YIELD: int
tickCounter: int

PotatoCrop()
grow(): void
getYield(): int
getTicksPerStage(): int
getCropKey(): String

TomatoCrop

TICKS_PER_STAGE: int
TOMATO_YIELD: int
tickCounter: int

TomatoCrop()
grow(): void
getYield(): int
getTicksPerStage(): int
getCropKey(): String

farm.core.GrowthStage

SEED
SPROUT
GROWING
MATURE
HARVESTED
DEAD

farm.core.FarmEvent

FRUITFUL_HARVEST
DROUGHT
BIRD_ATTACK
PEST

farm.core.FarmPlot

crop: Crop
soilQuality: int
isIrrigated: boolean
event: FarmEvent

daysSinceMature: int
cropJustVanished: boolean

FarmPlot()
plant(crop: Crop): void
tick(): void
getSoilQuality(): int
isIrrigated(): boolean
setIrrigated(irrigationStatus: boolean): void
applyEvent(e: FarmEvent): void
waterAm ount(): int
getCrop(): Crop
harvest(): int
centralBankInterferenceOrBrokeCitizenAlert(): void
getEvent(): FarmEvent
didCropJustVanish(): boolean

farm.core.IrrigationSystem

waterLevel: int
coverage: FarmPlot[]
COST_FOR_REFILL: int

IrrigationSystem(plots: FarmPlot[])
irrigateAll(): void
irrigateOne(p: FarmPlot): void
refill(): boolean
getW aterLevel(): int

farm.core.Farm

plots: FarmPlot[]
irrigation: IrrigationSystem
farmEvents: ArrayList<FarmEvent>

Farm(numberOfPlots: int)
addPlot(p: FarmPlot): void
tick(): void
activeEvents(): void
getActiveEvents(): ArrayList<FarmEvent>
getPlots(): FarmPlot[]
getPlot(i: int): FarmPlot
getNumberOfPlots(): int
getIrrigation(): IrrigationSystem
calculateProfitPerTick(): int

farm.core.FarmMarket

{final} inventory: M ap<S tring, Integer>
{final} priceIndex: M ap<String, Integer>

FarmMarket()
addToInventory(cropName: String, quantity: int, pricePerUnit: int): void
sellCrop(cropName: String, quantity: int): int
sellCrop(crop: Crop, quantity: int): int
getPriceOf(cropName: String): int
getPriceOf(crop: Crop): int
getInventory(): M ap<String, Integer>
getPriceIndex(): Map<String, Integer>
getVisitorsCapacity(): int

JFrame

FarmGUI.PlotPanel

{fin a l} in d e x: in t

paintComponent(g: Graphics): void
drawSoilBar(g2: Graphics2D, plot: FarmPlot, w: int, h: int): void
drawCenteredString(g2: Graphics2D, s: String, cx: int, cy: int, f: Font, c: Color): void

JPanel

GameInterface.MainMenu

GameInterface.NewGame GameInterface.LoadGame GameInterface.Credits GameInterface.MainMenu.FarmButton

JButton

SimulationInterface

view: CityView
simulator: Simulator

getBuildActionsPanel(city: City): JPanel
getBuildableClassesSortedByType(): ArrayList<Class<? extends Buildable>>

SimulationInterface.BottomPanel

SimulationInterface.ReactiveLabel SimulationInterface.Overlays SimulationInterface.DemandStats SimulationInterface.AggregateStats SimulationInterface.NumberStats SimulationInterface.TimeSpeedButtons SimulationInterface.Loans

T

JLabel

IsometricMapView

tiles: Im age[][]
cols: int
rows: int
tileW: int
tileH: int
zoom: double
surfaceOffset: int
originX: int
originY: int
hoverCol: int
hoverRow: int
drawHoverHighlight: boolean
dragState: int
dragStartCol: int
dragStartRow: int
dragCurCol: int
dragCurRow: int
controls: JPanel
controlsVisible: boolean
minZoom: double
maxZoom: double
zoomStep: double

IsometricMapView()
installKeyBindings(): void
buildControls(): void
panBy(dx: int, dy: int): void
zoomBy(factor: double): void
setZoomLimits(min: double, max: double): void
setControlsVisible(visible: boolean): void
isControlsVisible(): boolean
getControlsPanel(): JPanel
setTiles(grid: Im age[][]): void
setTile(col: int, row: int, image: Image): void
setTileSize(tileW idth: int, tileHeight: int): void
setO rigin(x: int, y: int): void
setZoom(zoom: double): void
getZoom(): double
setTileSurfaceOffset(offsetPixels: int): void
setHoverHighlightVisible(visible: boolean): void
getColumns(): int
getRows(): int
getTileW idth(): int
getTileHeight(): int
getHoveredCol(): int
getHoveredRow(): int
addTileHoverListener(l: TileHoverListener): void
removeTileHoverListener(l: TileHoverListener): void
addTileClickListener(l: TileClickListener): void
removeTileClickListener(l: TileClickListener): void
addTileDragListener(l: TileDragListener): void
removeTileDragListener(l: TileDragListener): void
addOverlay(o: OverlayPainter): Runnable
drawTileDiamond(g: Graphics2D, col: int, row: int, fill: Color, stroke: Color): void
attachComponent(comp: Component, col: int, row: int, anchor: TileAnchor): void
moveAttachment(comp: Component, col: int, row: int): void
detachComponent(comp: Component): void
addSprite(s: Sprite): void
removeSprite(s: Sprite): boolean
clearSprites(): void
getSprites(): List<Sprite>
tileToScreen(col: int, row: int): Point
tileAt(sx: int, sy: int): int[]
paintComponent(g: Graphics): void

IsometricMapView.TileAnchor JComponent IsometricMapView.TileHoverListener IsometricMapView.TileClickListener IsometricMapView.OverlayPainter IsometricMapView.TileDragListener

«static final»

IsometricMapView.Sprite

{final} im age: Im age
{final} anchorC ol: int
{final} anchorR ow: int
{final} footprintC ols: int
{final} footprintR ow s: int
{final} isO verlay: boolean

Sprite(image: Image, anchorCol: int, anchorRow: int, footprintCols: int, footprintRows: int)

«private static final»
IsometricMapView.Attachment

CityView

TILE_W: int
TILE_H: int
{final} CO LS: int
{final} RO W S: int
city: C ity
tick: int

CityView(city: City)
render(): void
buildView(): void
enableUpgradeAction(view: CityView, onClose: Runnable): Runnable
manhattanPath(from: Point, to: Point): List<Point>
loadImage(Class: Class<?>, filename: String): BufferedImage

«static»

CityView.DragVisual

CityView.TileListener CityView.FloatingDragLabelCreator CityView.FloatingHoverLabelCreator CityView.FloatingBuildingHoverLabelCreator CityView.TilesListener

CityView.ReactiveInfoLabel

labelCreator: Supplier<String>
render: Runnable

setLabelCreator(labelCreator: Supplier<String>): void

TogglesMenu

TogglesMenu(actions: List<Action>)

TogglesMenu.Action

getName(): String
getStopName(): String
enable(onEnd: Runnable): Runnable

1

*

1

*

1

*

1

1

1

*

0..1

1